

Version Java EE(Enterprise Edition) 8 (J2EE 1.8) maintained under Oracle / Jakarta EE 8 (maintained by eclipse foundation)

1. What is J2EE ?(Java Enterprise Edition)

Consists of specifications only .

Which specs ? (Rules or contract)

Specifications of primary essential services required for any enterprise application.

What is enterprise application ?

An enterprise application (EA) is a large software system platform designed to operate in a corporate environment . Typically , server side , transactional , multi threaded app.

It includes online shopping and payment processing, interactive product catalogs, computerized billing systems, security, content management, IT service management, business intelligence, human resource management, manufacturing, process automation, enterprise resource planning

These specifications include ---

|
Servlet API,JSP(Java server page) API,Security,Connection pooling ,EJB (Enterprise Java Bean), JNDI(Naming service -- Java naming & directory i/f),JPA(java persistence API),JMS(java messaging service),Java Mail, Java Server Faces , Java Transaction API, Webservices support(SOAP/REST) etc...

Vendor of J2EE specs -- Oracle / Sun / Eclipse

Implementation -- left to vendors (J2EE server vendors)

J2EE compliant web server --- Apache -- Tomcat (web server)

Services implemented --- servlet API,JSP API,Security,Connection pooling,JNDI(naming service)

J2EE complaint application server --- web container + EJB (enterprise java bean) container
+ ALL J2EE services implementation

J2EE server Vendors & Products

Apache -- Tomcat(web server) / Tomee (app server)

Oracle / Sun --- reference implementation --- Glassfish

Red Hat -- JBoss (wild fly)

Oracle / BEA -- Oracle weblogic

IBM -- Websphere

2. WHY J2EE

1. Can support different types of clnts --- thin client(web clnt)

thick clnt --- java application clnt(eg : TCP client)

smart clnts -- mobile clnts

2. J2EE server independence --- Create & deploy server side appln --on ANY J2EE compliant server --- guaranteed to produce SAME results w/o touching or re-deploying on ANY other J2EE server

3. Ready made implementation of primary services(eg --- security, conn,pooling,email....)--- so that J2EE developer DOESn't have to worry about primary services ---rather can concentrate on actual business logic.

3. Layers involved in HTTP request-response flow (refer to day1-data\day1_help\diags\request-response-flow.png)

Web browser sends the request (URL)

eg : http://www.abc.com:8080/day1.1

/day1.1 --- root / context path /web app name

Host --Web server--Web Container(server side JVM)--Web application---HTML/JSP/Servlet....

4. What is dyn web application --- server side appln --deployed on web server --- meant for servicing typically web clnts(thin) -- using application layer protocol HTTP /HTTPS
(ref : diag request-resp flow)

Read --HTTP basics including request & response structure from day1-data\day1_help\j2ee_prerequisites\HTTP Basics

URL : http://www.abc.com:8080/banking

http : app layer protocol(scheme)

www.abc.com : DNS qualified host name /IP address (to resolve the host)

8080 : TCP port no (to identify the port --> service : web server)

/banking : path or URI (uniform resource identifier)

In Java , when you are creating a web app

URI : starts with context path (web context =web app) , typically set as web project name.

5. Objective ? : Creating & deploying dyn web appln on Tomcat -- For HTML content

6. IDE automatically creates J2EE compliant web application folder structure .
Its details -- Refer to diag (J2EE compliant web app folder structure)

7. What is Web container --- (WC) & its jobs

1. Server side JVM residing within web server.

Its run-time environment for dynamic web components(Servlet & JSP,Filter) .

Jobs ---

1. Creating Http Request & Http response objects

2. Controlling life-cycle of dyn web comps (manages life cycle of servlet,JSP,Filters)

3. Giving ready-made support for services --- Naming,security,Conn pooling .

4. Handling concurrent request from multiple clients .

5. Managing session tracking

6. Manages state(session tracking)

7. page navigation

8. What is web.xml --- Deployment descriptor one per web appln

created by -- developer

who reads it -- WC

when --- @ deployment

what --- deployment instructions --- welcome page, servlet deployment tags, sess config, sec config.....

9. Why servlets? --- To add dynamic nature to the web application

What is a servlet ?

-- Java class (with NO main method) -- represents dynamic web component - whose life cycle will be managed by WC(web container : server side JVM)

life cycle methods --- init,service,destroy

Job list

1. Request processing
2. B.L (Business Logic)
3. Dynamic response generation
4. Data access logic(DAO class --managing DAO layer)
5. Page navigation

Servlet API details --refer to diag servlet-api.png

Objective 1: Test basic servlet life cycle -- init , service ,destroy

Creating & deploying Hello Servlet.

Deployment of the servlet

1. Via annotation

eg : `@WebServlet(value="/hello")`

public class HelloWorldServlet extends HttpServlet {...}

Meaning : A class level annotation , for WC(run time env) , to add the mapping between incoming url-pattern n the servlet.

Map : will be created by WC , at web app deployment time

key -- url-pattern eg : /hello

value -- Fully qualified servlet cls name (eg : pages.HelloWorldServlet)

URL : http://host:port/day1.1/hello

URI : /day1.1/hello

URL pattern : /hello

OR

2. Legacy approach : Using XML tags , in web.xml

How to deploy a servlet w/o annotations --- via XML tags

web.xml

```
<servlet>
  <servlet-name>abc</servlet-name>
  <servlet-class>pages.SecondServlet</servlet-class>
</servlet>
<servlet-mapping>
  <servlet-name>abc</servlet-name>
  <url-pattern>/test2</url-pattern>
</servlet-mapping>
```

WC : map

key : /test2

value : pages.SecondServlet

eg URL --http://host:port/day1_web/hello

At the time of web app deployment ---WC tries to populate map of url patterns , from XML tags (from web.xml). Later ---it will check for @WebServlet annotation

Objective 2: Test basic servlet life cycle -- init , service ,destroy (deployed via xml)

How to read request params sent from the client ?

javax.servlet.HttpServletRequest i/f methods

1. public String getParameter(String paramName) request.getParameter()

2. public String[] getParameterValues(String paramName)

Objective 3 : Accept different type of i/ps from user , in HTML form. Write a servlet to display request parameters.

OR

Start with a case study

Objective 3 .

What is default loading policy of WC for servlets ? lazy (i.e WC will start life cycle of the servlet :only after client sends the 1st request to the servlet)

Can you change it to eager ? Yes

Use Case : Typically for time consuming(heavy weight) inits.

eg : setting up DB conn, setting up spring framework....

HOW ?

@WebServlet (value="/test",loadOnStartup=1)

.....

OR

xml tag :

<load-on-startup>1</load-on-startup>

Solve -

Load HelloWorldServlet eagerly @ app deployment time.

Let HelloWorldServlet2 - get loaded by default , lazily !

Solve -

Load HelloWorldServlet eagerly @ app deployment time. - second

Let HelloWorldServlet2 - eagerly @ app deployment time. - first

Objective 4

Servlet JDBC Integration

case study - online voting

Complete user authentication

